

Managing Free Memory

1 Core Idea

Definition

Free-memory management is how the operating system records which parts of memory are free and which parts are occupied.

When memory is assigned dynamically, processes enter, leave, grow, and shrink. The operating system must quickly answer two questions:

- Which memory regions are free?
- Where can a new or swapped-in process fit?

Two Main Methods

Free memory is commonly tracked using **bitmaps** or **linked lists**.

2 Memory Picture

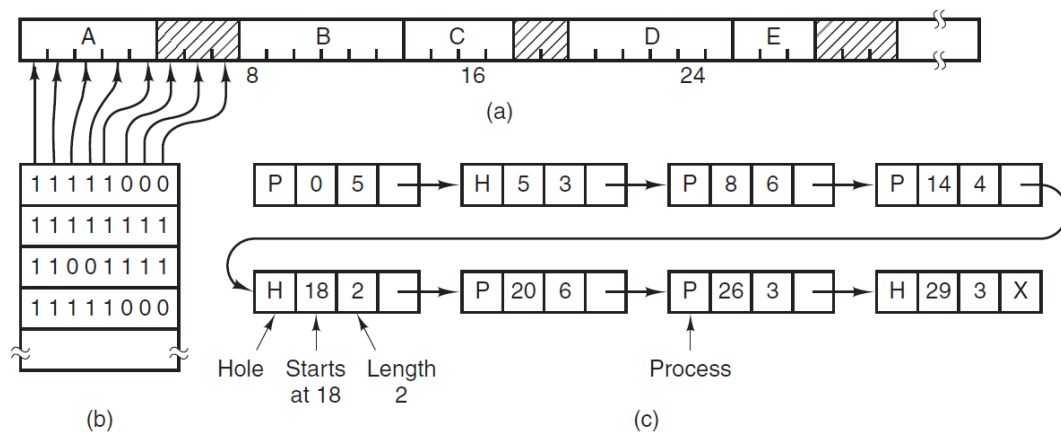


Figure 3-6. (a) A part of memory with five processes and three holes. The tick marks show the memory allocation units. The shaded regions (0 in the bitmap) are free. (b) The corresponding bitmap. (c) The same information as a list.

The figure shows the same memory state in three forms:

Part	Meaning
(a) Memory	Processes <i>A</i> to <i>E</i> occupy some regions; shaded regions are holes.
(b) Bitmap	Each allocation unit has one bit: free or occupied.
(c) Linked list	Each segment is stored as a process entry <i>P</i> or hole entry <i>H</i> .

What 8, 16, and 24 Mean

The numbers under the memory bar are **allocation-unit addresses**. They mark positions in memory, not process names or byte values.

For example, the label 8 means “allocation unit number 8.” Process *B* starts at unit 8, and process *C* starts at unit 14. The labels 8, 16, and 24 are shown as reference marks to make counting easier, like marks on a ruler.

Reading the linked-list entries

Each linked-list node in Fig. 3-6(c) has this form:

Type	Start	Length	Next
------	-------	--------	------

Field	Meaning
Type	<i>P</i> means process; <i>H</i> means hole.
Start	The allocation-unit number where the segment begins.
Length	Number of allocation units in the segment.
Next	Pointer to the next list entry; <i>X</i> means end of list.

Entry	How to read it
<i>P</i> 0 5	Process <i>A</i> starts at unit 0 and occupies units 0, 1, 2, 3, 4. So this is a process segment with start 0 and length 5.
<i>H</i> 5 3	The shaded hole after <i>A</i> starts at unit 5 and covers units 5, 6, 7. So this is a hole with start 5 and length 3.
<i>P</i> 8 6	Process <i>B</i> starts at unit 8 and occupies units 8 through 13. So this is a process segment with start 8 and length 6.
<i>H</i> 18 2	The hole after <i>C</i> starts at unit 18 and covers units 18, 19. So this is a hole with start 18 and length 2.

How to Count Length

The length is counted in allocation units, not bytes. For example, a segment covering units 18 and 19 has length 2, so its list entry is *H* 18 2.

3 Method 1: Bitmaps

Bitmap

A **bitmap** divides memory into fixed-size allocation units. Each unit has one bit in the map.

Bit value	Meaning
0	Unit is free.
1	Unit is occupied.

Some systems may reverse the meaning of 0 and 1. The important idea is that one bit describes one allocation unit.

4 Allocation Unit Size

The allocation unit size is a design choice.

Choice	Effect
Small allocation unit	Less internal waste, but a larger bitmap.
Large allocation unit	Smaller bitmap, but more wasted space in the last unit of a process.

Bitmap Size Idea

one allocation unit $\longleftrightarrow$ one bitmap bit

For example, if the allocation unit is 4 bytes, then 32 bits of memory need only 1 bitmap bit. So the bitmap uses about $1/32$ of memory.

5 Bitmap Strength and Weakness

Point	Explanation
Strength	Bitmap size is fixed once memory size and allocation-unit size are known.
Strength	Simple representation.
Weakness	To allocate a k -unit process, the OS must find k consecutive free bits.
Weakness	Searching for a long run of 0 bits can be slow.

Main Bitmap Cost

Finding a long consecutive free run is the expensive operation.

6 Method 2: Linked Lists

Linked-List Representation

A linked list stores memory as a sequence of segments. Each segment is either a process P or a hole H .

Each list entry stores:

- type: P for process or H for hole;
- starting address;
- length;
- pointer to the next segment.

Sorted by Address

Keeping the list sorted by address makes merging adjacent holes easier when a process exits or is swapped out.

7 When a Process Terminates

When process X terminates, its memory becomes free. The OS may need to merge neighboring holes.

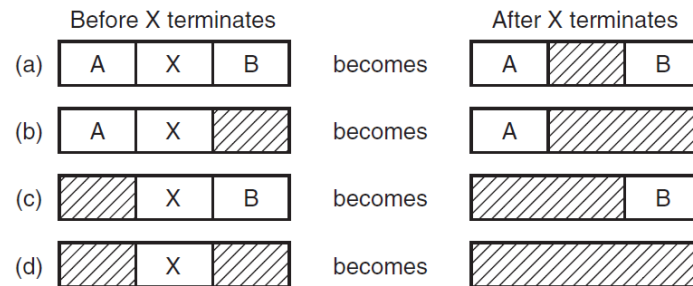


Figure 3-7. Four neighbor combinations for the terminating process, X .

Case	List update
(a) Process on both sides	Replace X 's process entry with a hole entry.
(b) Hole on the right	Merge X 's entry with the right hole.
(c) Hole on the left	Merge the left hole with X 's entry.
(d) Holes on both sides	Merge left hole, X , and right hole into one large hole.

Coalescing

Coalescing means combining adjacent free holes into one larger hole.

8 Single vs Double Linked List

A process table entry often points to the list entry for that process. When the process exits, the OS must check neighboring entries.

List type	Effect
Single linked list	Easy to find the next segment, harder to find the previous segment.
Double linked list	Easier to check both neighbors and merge holes.

9 Allocation Algorithms

When a process is created or swapped in, the memory manager must choose a hole large enough for it.

9.1 First Fit

First Fit

First fit scans from the start of the list and uses the first hole that is large enough.

If the hole is larger than needed, it is split into:

- the allocated process segment;
- the remaining smaller hole.

Why It Is Fast

First fit searches as little as possible.

9.2 Next Fit

Next Fit

Next fit is like first fit, but the next search starts where the previous search ended.

It avoids always starting from the beginning, but simulations show it can perform slightly worse than first fit.

9.3 Best Fit

Best Fit

Best fit searches the entire list and chooses the smallest hole that is large enough.

Best fit tries to avoid wasting a large hole on a small request.

Surprising Problem

Best fit often creates many tiny holes that are too small to be useful.

9.4 Worst Fit

Worst Fit

Worst fit chooses the largest available hole.

The goal is to leave a remaining hole that is still large enough to be useful. In practice, simulations show that worst fit is usually not a good strategy.

10 Example: First Fit vs Best Fit

Using the memory in Fig. 3-6, suppose a block of size 2 is requested.

Algorithm	Choice
First fit	Chooses the first large enough hole, starting at 5.
Best fit	Chooses the smallest sufficient hole, starting at 18.

11 Separate Lists for Processes and Holes

Allocation can be faster if the OS keeps separate lists:

- one list for processes;
- one list for holes.

Benefit	Cost
Allocation scans only holes.	Deallocation becomes more complex.
Best fit can be faster if holes are sorted by size.	Finding neighbors for merging becomes harder.

If the hole list is sorted by size, best fit becomes fast: the first hole that fits is already the smallest adequate hole.

12 Storing List Data Inside Holes

When holes have their own list, the OS can store hole metadata inside the hole itself.

Hole word	Stores
First word	Hole size.
Second word	Pointer to the next hole.

This avoids separate list nodes for holes.

13 Quick Fit

Quick Fit

Quick fit keeps separate lists for common hole sizes, such as 4 KB, 8 KB, 12 KB, and so on.

Finding a hole of a common size becomes very fast.

Quick Fit Problem

When a process terminates, finding neighboring holes for merging can be expensive. Without merging, memory quickly fragments into many small holes.

14 Summary Table

Method	Main idea	Main issue
Bitmap	One bit per allocation unit.	Searching for long free runs is slow.
Linked list	Track process and hole segments.	Must update and merge entries.
First fit	Use first adequate hole.	May leave scattered holes.
Best fit	Use smallest adequate hole.	Creates many tiny holes.
Worst fit	Use largest hole.	Usually poor performance.
Quick fit	Lists for common sizes.	Fast allocation, harder merging.